Itnavideo — Faceless Video Generator Audit

The specification is a solid foundation for a focused, 16:9 “faceless documentary” generator, but it is currently **feature-thin, workflow-incomplete, and operationally underspecified** for a production AI video platform. The most important gaps are: no editable scene/storyboard stage, no asset licensing/rights controls, weak mobile treatment, no render lifecycle/status model, and insufficient validation, observability, and failure recovery around the AI and Lambda pipeline. Remotion Lambda is designed for parallelized rendering with explicit concurrency and timeout controls, so the spec’s “clamp to 9,000 frames” approach addresses memory but not render reliability, cost, retries, or user-visible progress.

Executive assessment

Area	Current state	Severity	Required outcome
Product workflow	Four steps, but no editable storyboard or visual review	Critical	Add a review/edit stage before render
AI planning	“Semantic scene beats” is vague; no schema, confidence, or correction model	Critical	Define deterministic scene schema, validation, and user editing

Stock footage	“Curated” and “keyword matched,” but no licensing, rejection, or replacement flow	Critical	Add licensed-source policy, visual QA, and per-scene replacement
Audio	Transcription and ducking defined; no loudness, alignment, or speaker handling	High	Add normalization, word-level timing, and audio QA
UI/UX desktop	Basic 3-row layout; no state model, empty/error states, or editing interactions	High	Build a persistent, editable studio workspace
Mobile	“Responsive” is mentioned but no mobile-specific behavior is specified	High	Create a mobile-first task flow and preview strategy
Rendering	Lambda render is mentioned, but no job lifecycle, retries, artifacts, or delivery	Critical	Implement durable render pipeline and status UX
QA	Only three checks; no end-to-end, accessibility, performance, or visual regression tests	Critical	Define test matrix and acceptance criteria
Accessibility	No accessibility requirements despite subtitles and video output	High	Meet WCAG 2.2 AA for UI and caption quality

WCAG 2.2 requires sufficient contrast for text and non-text UI, and accessible names/states for controls; these need to become explicit implementation requirements rather than assumptions.

Critical product gaps

1. Add an editable storyboard stage

The current flow jumps from transcript/music/subtitle selection directly to preview and render. This is the largest product weakness: users cannot correct an incorrect scene visual, remove an irrelevant B-roll clip, change a diagram, reorder scenes, or verify that AI-generated SVG content is factually and visually appropriate.

What to change

Insert a mandatory **Scene Review & Storyboard** step between Step 3 and Step 4:

1. Upload/transcribe.
2. Music/audio design.
3. Subtitles and pacing.
4. **Storyboard review and editing.**
5. Preview and render.

Required storyboard capabilities

- Display each scene as a card containing:
    - Start/end time.
    - Narration excerpt.
    - Generated headline or on-screen text.
    - Visual type: stock footage, SVG diagram, vignette, kinetic typography.
    - Thumbnail or live preview.
    - Selected stock asset.
    - Confidence/status indicator.
- Allow users to:
    - Replace stock footage.
    - Regenerate a scene visual.

    - Edit on-screen headline and subtitle text.
    - Change visual layout: split screen, full-bleed, diagram, quote, metric, comparison.
    - Reorder scenes.
    - Delete or merge scenes.
    - Lock a scene so later regeneration does not overwrite it.
    - Mark a scene as approved.
- Show an overall completion indicator: “12 of 14 scenes approved.”
- Prevent render if unresolved critical validation errors exist, while allowing non-blocking warnings.

Why

AI-generated visual planning will never be perfect. A user must be able to intervene before spending credits and waiting for a cloud render. This also reduces support burden, refunds, and low-quality output.

Implementation for Antigravity

- Extend the existing `FacelessVideoStudio.tsx` state model with a `storyboard` collection.
- Store each scene as a versioned object:
    - `sceneId`
    - `startTime`
    - `endTime`
    - `narrationText`
    - `visualType`
    - `visualPrompt`
    - `selectedAssetId`
    - `svgLayout`
    - `headline`
    - `subtitleCues`
    - `status`: `draft`, `generating`, `ready`, `warning`, `error`, `approved`
    - `locked`
    - `revision`
- Persist storyboard drafts server-side keyed by `userId + projectId`, with autosave and optimistic UI updates.

- Use a queue or asynchronous job for “regenerate scene”; never block the entire UI.
- Render preview thumbnails from the same Remotion scene props where feasible, or use lightweight static previews for low-cost interaction.

2. Define AI planning as a contract, not a feature label

“Gemini parses narration into structured scenes with SVG primitives” is insufficient for implementation. The specification does not define the output schema, validation rules, visual grounding rules, hallucination prevention, or fallback behavior beyond a generic deterministic storyboard.

What to change

Specify an AI Planning Contract with:

- Input:
  - Cleaned transcript.
  - Language detection result.
  - Audio duration.
  - User-selected pacing.
  - Existing user edits/locks.
  - Content policy and brand constraints.
- Output:
  - Structured JSON scenes.
  - Scene-level visual intent.
  - Search keywords and negative keywords.
  - SVG layout type.
  - Data points extracted from transcript, only when explicitly supported.
  - Subtitle cues with word-level or phrase-level timing.
  - Confidence score.
  - Reasoning trace or brief rationale for internal QA/debugging.
- Validation:
  - JSON Schema validation.
  - Time continuity validation: no gaps or overlaps.
  - Scene duration validation: 3–6 seconds, with permitted exceptions.
  - Asset search-query safety validation.
  - Prohibition of unsupported numeric claims in diagrams.

- Fallback storyboard generation if validation fails.

Why

Without a strict contract, model changes will break the Remotion template, create malformed visuals, and make debugging impossible.

Implementation for Antigravity

- Create a shared Zod schema used by:
  1. AI planning service.
  2. API response validation.
  3. Remotion props generation.
  4. Storyboard editor.
- Use a two-stage pipeline:
  1. **Plan**: transcript to scene outline.
  2. **Visualize**: scene outline to visual assets, SVG layout, and subtitle cues.
- Cache planning results by content hash, model version, prompt version, and user selections.
- Add a `planningVersion` field to every project so old renders remain reproducible.
- Add deterministic fallback only after structured validation fails; do not silently replace a valid AI plan.

3. Make stock footage licensing and quality explicit

The spec says “curated cloud stock footage,” but does not identify sources, license type, attribution requirements, expiration, geographic restrictions, or whether assets can be used commercially. For a product explicitly aimed at monetized YouTube channels, this is a legal and business risk.

What to change

Add a dedicated **Media Asset Policy**:

- Approved providers and license types.
- Commercial-use eligibility.
- Attribution requirements, if any.

- Asset expiration and re-validation policy.
- Duplicate-asset prevention across scenes.
- Explicit prohibition on AI-generated or scraped assets unless separately approved.
- Per-asset metadata: provider, license ID, resolution, duration, orientation, model releases where applicable, and license expiry.
- Automatic fallback asset pool if a selected asset becomes unavailable.
- User-facing license indicator: “Commercial use included.”

Why

A video that renders successfully can still be unusable if its footage license is invalid.

This must be enforced at asset selection, render submission, and final delivery.

Implementation for Antigravity

- Add an `assets` table/collection with immutable license metadata.
- Add an `assetUsage` relation to each scene/render.
- Before render, run a preflight validation:
  - Asset exists.
  - License is active.
  - Asset resolution meets 1920×1080 requirement.
  - Asset is not expired.
  - Asset is available in the render region.
- If validation fails, block render and offer equivalent replacements.
- Store render-time asset snapshot IDs so a delivered video can be audited later.

4. Add content safety, factual-risk, and brand controls

The product targets educational and documentary content, where inaccurate diagrams, misleading charts, and unsafe stock imagery can create serious credibility and policy issues.

What to change

Add:

- Content policy screening for prohibited or unsafe visual requests.
- Stock-footage safety checks.
- Diagram data validation: no invented statistics, percentages, timelines, or causal claims.
- Source attribution or “illustrative” labeling for non-verified data visuals.
- Optional user-provided fact source or reference URL.
- Brand controls:
    - Logo upload.
    - Color palette.
    - Font selection from licensed fonts.
    - Intro/outro templates.
    - Channel watermark.
    - Safe-area guides.

Why

“Procedural SVG diagrams grounded in the transcript” can easily become fabricated data visualization if the transcript does not actually contain usable figures.

Implementation for Antigravity

- Add a `dataEvidence` field to diagram-type scenes.
- If no numeric evidence exists, force the scene into a conceptual layout: icons, labels, sequence, comparison without quantitative axes, or quote card.
- Add a visual QA pass using a multimodal model or deterministic rules:
    - Detect unreadable text.
    - Detect clipped elements.
    - Detect low-contrast text.
    - Detect inappropriate imagery.
    - Detect empty/broken SVG areas.
- Surface issues as actionable warnings in the storyboard editor.

Workflow improvements

Current workflow weaknesses

Current step	Gap	Recommended replacement
Upload audio or enter title	“Enters a script title” is ambiguous; no text-to-speech path is defined	Support audio upload, pasted script, uploaded script file, and optional AI voiceover
Transcription review	No alignment, speaker, language, or correction workflow	Provide transcript editor with audio playback, speaker labels, language confirmation, and re-transcribe
Music selection	Only volume and ducking; no mood fit, length, fade, or preview synchronization	Add music preview, auto-trim/loop, fade in/out, ducking sensitivity, and music replacement
Subtitle style	32 presets but no readability, safe-area, or timing controls	Add preset preview, font size, position, contrast, max words per line, and timing adjustment
Visual pacing	“Motion pacing” is undefined	Define slow, standard, dynamic presets and per-scene override
Preview	“Live cinema preview” is vague	Define real-time preview scope, playback controls, scrubbing, and what is previewed versus final-rendered

Render	One button, no cost confirmation or job tracking	Add estimate, confirmation, queue position, progress, cancel/retry, and delivery

Recommended end-to-end flow

1. **Create project**
   - Choose Faceless Video.
   - Enter topic/title.
   - Choose input method: audio, script, or both.
2. **Prepare narration**
   - Upload audio or paste script.
   - If script-only, offer optional AI voice selection or require user-supplied voiceover.
   - Detect language.
   - Transcribe and align audio.
   - Let user edit transcript.
   - Show duration, word count, estimated scenes, and estimated credits.
3. **Configure style**
   - Music mood and track.
   - Music volume and ducking.
   - Subtitle preset.
   - Subtitle readability controls.
   - Transition style.
   - Pacing.
   - Brand assets.
4. **Generate storyboard**
   - Show planning progress.
   - Display scenes.
   - Allow editing and regeneration.
   - Show asset previews and warnings.
5. **Review**
   - Full timeline preview.
   - Scene-by-scene review.
   - Audio/video sync check.
   - Mobile/desktop safe-area check.
   - Accessibility check for subtitles.

6. **Confirm and render**
    - Show exact duration and credits.
    - Obtain explicit render confirmation.
    - Start render job.
    - Show progress and allow user to leave the studio.
7. **Deliver**
    - Download MP4.
    - Download SRT/VTT captions.
    - Download thumbnail(s).
    - View render history.
    - Re-render after edits, with clear credit implications.

Missing functional requirements

Input and narration

The specification needs to resolve a major ambiguity: can a user create a video from **text only**? It says users may “enter a script title,” but does not state whether the platform generates narration, accepts text-to-speech, or requires uploaded audio.

Required decisions

- Support these explicit input modes:
    - Audio-only.
    - Script-only with AI voiceover.
    - Script plus uploaded voiceover.
    - Audio plus optional transcript correction.
- Add voice selection:
    - Gender, language, accent, tone, speed, and preview.
    - Clear disclosure when a voice is AI-generated.
- Add script length validation:
    - Minimum and maximum characters/words.
    - Estimated spoken duration.
    - Warning when script duration exceeds 300 seconds.
- Support multilingual input more explicitly:
    - Hindi, Hinglish, English, and other regional languages.
    - Clear statement of subtitle script and translation behavior.
    - User ability to choose subtitle language separately from spoken language.

Implementation

- Add `inputMode`: `audio`, `script`, `audio_plus_script`.
- Add `voiceConfig`: `voiceId`, `language`, `speed`, `pitch`, `style`.
- Add transcript normalization and alignment service.
- Store both original transcript and user-edited transcript.
- Recalculate scene boundaries after material transcript edits.

Duration and credits

The billing rule is mathematically clear, but incomplete in product terms.

Required additions

- Define whether credits are charged at:
  - Render submission.
  - Render success.
  - Render failure.
- Define behavior for:
  - Failed render.
  - Partial render.
  - User cancellation.
  - Re-render after minor edit.
  - Duplicate render request.
  - Concurrent render limit.
- Show a pre-render estimate:
  - Exact duration in seconds.
  - Credit cost.
  - Expected render time range.
  - Output resolution and file format.
- Prevent rendering when:
  - Duration exceeds 300 seconds.
  - User lacks credits.
  - Required assets are unavailable.
  - Audio is missing or invalid.
  - Storyboard has unresolved errors.

Implementation

- Use an idempotency key for render submission.

- Create a render job record before invoking Lambda.
- Reserve credits at submission; finalize or refund on terminal job state.
- Add `renderAttempts`, `renderErrorCode`, `renderDurationMs`, `outputUrl`, `thumbnailUrl`, and `creditTransactionId`.
- Use server-side duration calculation; never trust client-side duration for billing.

Output and delivery

The spec defines MP4 output but not the complete delivery package.

Required outputs

- 1920×1080, 30 FPS MP4.
- Configurable bitrate/quality tier, if applicable.
- Optional 720p preview or social variant, if product strategy requires it.
- SRT and VTT subtitle files.
- Poster/thumbnail image.
- Optional audio-only file.
- Project JSON export for reproducibility.
- Render manifest with template version, asset IDs, model versions, and prompt versions.

Why

Creators need captions and thumbnails for YouTube. A reproducible manifest also enables debugging and future re-renders.

Desktop UI/UX improvements

Layout

The proposed three-row layout is reasonable, but it needs a more robust studio workspace.

Recommended desktop layout

- **Left rail**: Project navigation, steps, autosave status, warnings.
- **Center canvas**: Preview and active editor.
- **Right inspector**: Contextual properties for selected scene, music, subtitle, or render settings.
- **Bottom timeline**: Scene blocks, audio waveform, subtitle cues, music track, and transitions.
- **Sticky action bar**: Primary CTA, credit balance, duration, validation status, and help.

This is more intuitive than forcing users through a rigid four-step wizard, because video creation is inherently iterative.

Essential interactions

- Autosave with visible “Saved” / “Saving” / “Offline” state.
- Undo/redo for storyboard edits.
- Keyboard shortcuts:
  - Space: play/pause.
  - Arrow keys: scrub.
  - `J` / `L`: backward/forward.
  - `Cmd/Ctrl + Z`: undo.
  - `Cmd/Ctrl + Shift + Z`: redo.
- Drag-to-reorder scenes.
- Inline editing for text.
- Scene replacement through a searchable asset drawer.
- Split view: preview on left, storyboard/editor on right.
- Persistent project state when navigating away.
- Beforeunload warning only when there are unsaved critical changes.
- Empty, loading, error, partial-success, and offline states for every panel.

Preview requirements

“Live Cinema Preview” must be precisely defined.

Required preview behavior

- Play/pause.
- Scrub timeline.
- Jump to previous/next scene.

- Mute/unmute.
- Volume control.
- Fullscreen.
- Playback speed: 0.5x, 1x, 1.5x, 2x.
- Scene markers.
- Subtitle toggle.
- Music-only preview.
- Narration-only preview.
- Safe-area overlay.
- “Approximate preview” disclosure where browser preview differs from final Lambda render.

Why

Users must understand what they are approving. A browser preview cannot always perfectly match cloud rendering, especially for fonts, complex effects, and asset processing.

Mobile UI/UX improvements

The specification currently treats mobile as an afterthought. For a creation tool, mobile should support review, monitoring, and lightweight editing—not necessarily full complex editing.

Mobile-first strategy

Define two mobile experiences:

1. **Mobile creation and monitoring**
    - Upload audio.
    - Paste script.
    - Choose music and subtitle style.
    - Review storyboard.
    - Approve scenes.
    - Start render.
    - Track render progress.

    - Download final video.
2. **Mobile-optimized editing**
    - Edit subtitles and headlines.
    - Replace stock footage.
    - Regenerate a scene.
    - Reorder scenes.
    - Adjust music volume.
    - View warnings.

Avoid expecting precise timeline editing, SVG manipulation, or multi-track editing on small screens.

Mobile layout

- Use a single-column, card-based flow.
- Make the preview sticky at the top or use a bottom sheet.
- Use bottom navigation or a step progress bar.
- Use large touch targets: minimum 44×44 CSS pixels.
- Avoid hover-only controls.
- Use native file pickers for uploads.
- Show upload progress, cancel, retry, and background-upload behavior.
- Use swipeable scene cards.
- Use bottom sheets for asset selection and scene settings.
- Ensure the render CTA remains accessible without covering the preview.
- Support both portrait and landscape orientations.
- Handle keyboard opening without obscuring the active input.

Mobile-specific edge cases

- Interrupted upload due to network loss.
- Backgrounded transcription.
- Render started on mobile but completed later.
- Push/email notification when render completes.
- Low-storage warning before download.
- Cellular-data warning for large uploads/downloads.
- Reduced-motion preference.
- Screen-reader labels for scene cards, timeline markers, and status changes.

Technical architecture gaps

Render pipeline

The current spec says Lambda rendering occurs, but it does not define the operational architecture. “Frame ranges are clamped to 9,000 frames” prevents one class of failure but does not make the system reliable.

Required architecture

- API receives render request and validates it.
- API creates a durable render job.
- API reserves credits.
- Job queue dispatches render worker.
- Render worker invokes Remotion Lambda.
- Render progress is written to a job store.
- WebSocket, polling, or server-sent events update the client.
- On completion, output is uploaded to durable object storage.
- Signed download URLs are issued.
- Failure triggers structured error capture and retry policy.
- Stale jobs are reconciled by a periodic sweeper.

Why

Remotion Lambda parallelizes rendering across multiple Lambda invocations; concurrency, account limits, timeouts, and storage constraints must be managed explicitly.

Implementation for Antigravity

- Add `renderJob` entity:
  - `jobId`
  - `projectId`
  - `userId`
  - `status`
  - `progress`
  - `framesTotal`
  - `framesCompleted`
  - `startedAt`

    - `completedAt`
    - `outputUrl`
    - `errorMessage`
    - `errorCode`
    - `retryCount`
    - `idempotencyKey`
    - `templateVersion`
    - `remotionCompositionVersion`
- Use a queue-backed worker rather than invoking Lambda directly from the browser-facing API.
- Implement exponential backoff with a maximum retry count.
- Make render submission idempotent.
- Store render artifacts in versioned cloud storage.
- Clean up intermediate artifacts after a defined retention period.
- Add a dead-letter queue for permanently failed jobs.

Lambda configuration

The specification's "Lambda render timeout" language is imprecise. Remotion recommends relatively short Lambda function timeouts and higher concurrency rather than one long-running function; its production guidance specifically notes a default timeout around 120 seconds and recommends adjusting based on measured needs.

Required changes

- Replace "Lambda render timeout" with:
    - `functionTimeout`
    - `renderTimeout`
    - `delayRenderTimeout`
    - `queueVisibilityTimeout`
    - `clientProgressTimeout`
- Define concurrency policy:
    - Per-user concurrent render limit.
    - Account-wide concurrency ceiling.
    - Queue-based admission control.
    - Retry budget.
- Monitor:
    - Queue depth.

        - Render success rate.
        - P50/P95 render duration.
        - Lambda errors.
        - Throttling events.
        - Cost per render.
        - Storage growth.
    - Set alerts for:
        - Render failure rate.
        - Stuck jobs.
        - Lambda concurrency saturation.
        - Storage quota.
        - Credit reconciliation mismatch.

Implementation

- Configure Remotion with explicit `concurrency` or `framesPerLambda`, not an arbitrary frame clamp alone.
- Keep `framesPerLambda` within Remotion's documented bounds; Remotion identifies a maximum of 200 Lambda functions per render and recommends at least 20 frames per Lambda as a baseline.
- Use `renderMediaOnLambda` with explicit timeout and error callbacks.
- Preserve the 9,000-frame cap as a product constraint, but do not treat it as the only reliability control.

Audio pipeline

The audio specification is incomplete. "Auto-ducking to 12%" is a useful default, but it does not ensure professional-sounding output.

Required audio requirements

- Loudness normalization using EBU R128-style loudness measurement; FFmpeg's `loudnorm` filter implements this approach, and two-pass normalization is recommended for more accurate results.
- Target loudness and true-peak limits should be defined.
- Voice/music separation or voice-activity detection.
- Ducking attack/release times.
- Music fade-in and fade-out.
- Music looping or trimming behavior.
- Audio drift detection.

- Transcode to a consistent internal format before scene planning.
- Preserve original audio for re-rendering.

Implementation

- Normalize narration before transcription and again as part of final mix preparation.
- Use a loudness target appropriate for spoken YouTube content, for example approximately -16 LUFS integrated, with a defined true-peak ceiling.
- Implement ducking based on voice activity, not only scene boundaries.
- Add audio QA:
    - Silence detection.
    - Clipping detection.
    - Excessive noise detection.
    - Missing audio detection.
    - Drift between transcript timing and audio.

Error handling and edge cases

The current error section covers only three cases. A production system needs a complete failure matrix.

Required edge cases

Area	Edge case	Required behavior
Upload	Unsupported file type	Clear validation before upload
Upload	File too large	Show size limit and compression guidance
Upload	Corrupt audio	Detect and prompt re-upload

Upload	Upload interrupted	Resume or restart cleanly
Transcription	No speech	Current behavior is correct; retain no-credit rule
Transcription	Partial speech	Warn user and allow re-transcription
Transcription	Wrong language detected	Let user override language
Transcription	Low-confidence words	Highlight uncertain words for review
AI planning	Malformed JSON	Validate, retry once, then deterministic fallback
AI planning	Empty transcript	Block planning with actionable message
AI planning	Contradictory scene timing	Repair or reject before UI display

Stock assets	No relevant footage	Offer SVG, vignette, typography, or user-upload alternative
Stock assets	License expired	Block render and suggest replacement
Stock assets	Asset removed from provider	Replace or require user action
SVG	Text overflow	Auto-scale, wrap, or simplify layout
SVG	Invalid generated data	Reject unsupported data visualization
Preview	Browser codec unsupported	Provide fallback preview or clear browser guidance
Render	Insufficient credits	Block before submission
Render	Concurrent limit reached	Show queue position or scheduled render
Render	Lambda throttling	Retry with backoff and user-visible status

Render	Partial failure	Retry failed chunks if supported; otherwise fail safely
Render	User closes browser	Continue render server-side
Delivery	Expired download URL	Allow regeneration of signed URL
Delivery	Deleted project	Retain or explicitly purge outputs per retention policy

Accessibility requirements

Accessibility must be treated as a first-class requirement.

Required UI accessibility

- WCAG 2.2 AA conformance for the studio interface.
- Visible keyboard focus states.
- Logical focus order.
- Accessible names for all icon-only buttons.
- `aria-live` announcements for:
  - Upload complete.
  - Transcription complete.
  - Scene generation complete.
  - Render progress milestones.
  - Render success/failure.
- No color-only status communication.
- Minimum contrast for text and meaningful UI elements; WCAG 2.2 specifies at least 4.5:1 for normal text, with separate provisions for large text.
- Support `prefers-reduced-motion`.
- Avoid seizure-inducing transitions.

- Provide captions and transcript access in the preview.
- Ensure subtitle presets meet minimum contrast and readability standards.

Required output accessibility

- Burned-in subtitles must have:
  - High contrast.
  - Safe margins.
  - Limited characters per line.
  - Sufficient display duration.
- Provide downloadable SRT/VTT.
- Avoid conveying essential information through color alone.
- Provide alt text or textual descriptions where the product offers shareable preview assets.

Testing requirements

The current QA section is inadequate for a customer-facing AI video generator.

Required test layers

Layer	Tests
Unit	Credit calculation, duration validation, scene schema validation, subtitle cue generation, audio ducking parameters
Integration	Upload, transcription, planning, asset selection, storyboard persistence, render submission
Contract	Gemini response schema, Groq/Gemini transcription response, asset-provider API, storage API

E2E	Audio-to-render, script-to-render, edit-and-re-render, failed-render recovery
Visual regression	Subtitle presets, SVG layouts, transitions, safe areas, 1080p output
Performance	30-second, 2-minute, and 5-minute projects; large transcripts; asset-heavy scenes
Reliability	Lambda throttling, network interruption, retry, idempotent submission, stale job recovery
Accessibility	Keyboard navigation, screen-reader announcements, focus visibility, reduced motion
Security	File-type validation, malware scanning, signed URL expiry, authorization, credit fraud prevention
Localization	Hindi, Hinglish, English; RTL readiness if applicable; subtitle wrapping

Required acceptance criteria

- A 125-second video bills exactly 3 credits.
- A 300-second video bills exactly 5 credits.
- A 301-second project cannot be rendered.
- A failed render does not permanently consume credits unless product policy explicitly defines otherwise.

- A user can edit a storyboard after AI generation.
- A user can replace an unsuitable stock asset.
- A user can render after closing and reopening the browser.
- A render job survives a Lambda invocation failure through retry or clear failure state.
- Every generated SVG diagram either has transcript-supported data or is explicitly labeled as conceptual.
- Every render produces MP4, SRT/VTT, thumbnail, and reproducible manifest.
- Keyboard-only users can complete the full creation flow.
- Mobile users can upload, review, approve, render, and download without desktop-only dependencies.

Implementation instructions for Antigravity

Implement the following changes in priority order, preserving the existing `facelessVideo` / `FACELESS_VIDEO` architecture and keeping it separate from the 9:16 Shorts pipeline.

Priority 1: Workflow and state

1. Add a fifth workflow stage: **Storyboard Review & Edit**.
2. Refactor `FacelessVideoStudio.tsx` into a state-driven studio with:
    - Project state.
    - Transcript state.
    - Audio state.
    - Style state.
    - Storyboard state.
    - Render state.
    - Validation state.
3. Add server-side project persistence with autosave.
4. Add optimistic updates and undo/redo for storyboard edits.
5. Add a persistent timeline and scene inspector.
6. Add explicit render confirmation showing duration, credits, and unresolved issues.

Priority 2: AI planning contract

1. Define a versioned Zod schema for the AI planning output.

2. Split planning into:
    - Transcript-to-scene-outline.
    - Scene-outline-to-visual-plan.
3. Add JSON Schema/Zod validation.
4. Add deterministic repair and fallback only after validation failure.
5. Add confidence scores and warning reasons.
6. Add scene regeneration with user locks.
7. Add a `planningVersion` and `promptVersion` to every project and render manifest.

Priority 3: Asset governance

1. Add an asset registry with license, provider, resolution, expiry, and commercial-use metadata.
2. Add pre-render asset validation.
3. Add replacement flow for unavailable, low-quality, or unsuitable assets.
4. Add fallback visual modes: SVG, vignette, typography, user upload.
5. Add visual QA for readability, clipping, contrast, and unsafe imagery.
6. Store asset usage in the render manifest.

Priority 4: Render reliability

1. Replace direct client-to-Lambda assumptions with a durable job queue and worker.
2. Add `renderJob` persistence, idempotency keys, retries, progress, cancellation, and terminal-state reconciliation.
3. Configure explicit Remotion Lambda concurrency/`framesPerLambda`, timeout, retry, and monitoring.
4. Keep the 9,000-frame cap as a product limit, but add proper queue and concurrency controls.
5. Add progress events and background completion notifications.
6. Store final MP4, captions, thumbnail, and render manifest in durable storage.

Priority 5: Audio quality

1. Add audio normalization using EBU R128-style loudness processing.
2. Add voice activity detection and time-based ducking.
3. Add fade-in, fade-out, music trim/loop, and ducking sensitivity.
4. Add audio QA for clipping, silence, noise, and drift.

5. Add optional AI voiceover only if product decision confirms it; otherwise clearly require uploaded narration.

Priority 6: UI/UX

1. Build a desktop three-pane studio: left navigation, center preview, right inspector.
2. Add a bottom timeline with scene, audio, subtitle, and music layers.
3. Add mobile single-column flow with bottom sheets and swipeable scene cards.
4. Add complete empty, loading, error, warning, offline, and partial-success states.
5. Add keyboard shortcuts, focus management, and `aria-live` status updates.
6. Add safe-area overlays and subtitle readability controls.

Priority 7: QA and observability

1. Add unit, integration, contract, E2E, visual regression, accessibility, performance, and reliability tests.
2. Add structured logging with `projectId`, `userId`, `sceneId`, `renderJobId`, and `assetId`.
3. Add metrics for transcription success, planning success, asset match rate, render success, render duration, cost, and credit reconciliation.
4. Add alerting for stuck jobs, elevated failure rates, Lambda throttling, and credit anomalies.
5. Add a QA checklist gate before each template or model change.

Final specification additions

Add the following explicit requirements to the specification before implementation:

- **Input modes**: Audio-only, script-only with optional AI voice, and script plus uploaded voiceover.
- **Storyboard editing**: Mandatory review and editing before render.
- **Scene schema**: Versioned, validated JSON contract for every generated scene.
- **Asset licensing**: Commercial-use validation and auditable license metadata.
- **Render lifecycle**: Submitted, queued, rendering, progress, succeeded, failed, canceled, expired.
- **Credit lifecycle**: Reserve on submission, settle on success, refund or define policy on failure.
- **Audio mastering**: Loudness normalization, voice activity detection, ducking attack/release, fades, and drift detection.

- **Output package**: MP4, SRT/VTT, thumbnail, project JSON, and render manifest.
- **Mobile experience**: Mobile-first creation, review, monitoring, and lightweight editing.
- **Accessibility**: WCAG 2.2 AA, keyboard support, screen-reader announcements, reduced motion, and caption readability.
- **Observability**: Structured logs, metrics, alerts, job reconciliation, and cost tracking.
- **Testing**: Full test matrix beyond typechecking and two isolated checks.

The single most important change is to make the product **editable before render**, not merely generative. Without storyboard review, asset replacement, validation, and a reliable render pipeline, the current specification will produce a technically functional but operationally fragile and commercially risky video generator. Remotion's own production guidance emphasizes memory tuning, bounded concurrency, appropriate timeouts, and awareness of account/storage limits—requirements the current spec does not yet operationalize.

Video-spec architecture review: turn the Itnavideo MD spec into a production-ready component blueprint, QA test matrix, and UX audit
Computer